Mount Kenya University

Unlocking Infinite Possibilities

Student Name: Kenan Kiplimo

Admission Number: BSCCS/2025/67071

Course/Class: Bachelor of Science in Computer science

Unit Code: BIT4138

Unit Name: Advanced Cryptography

Lecturer Name: Mr. Michael Nyoro

Semester/Academic Year: Year 3 Semester 2, May 2026

Selected Technologies:

- ParrotOS Security
- VSCodium
- Python

GitHub: https://github.com/Kynan51/advanced-cryptography

Week 1: Cryptography Environment Setup

Installation and Configuration of Cryptography Development Environment

Fig 1: Installation of VSCodium

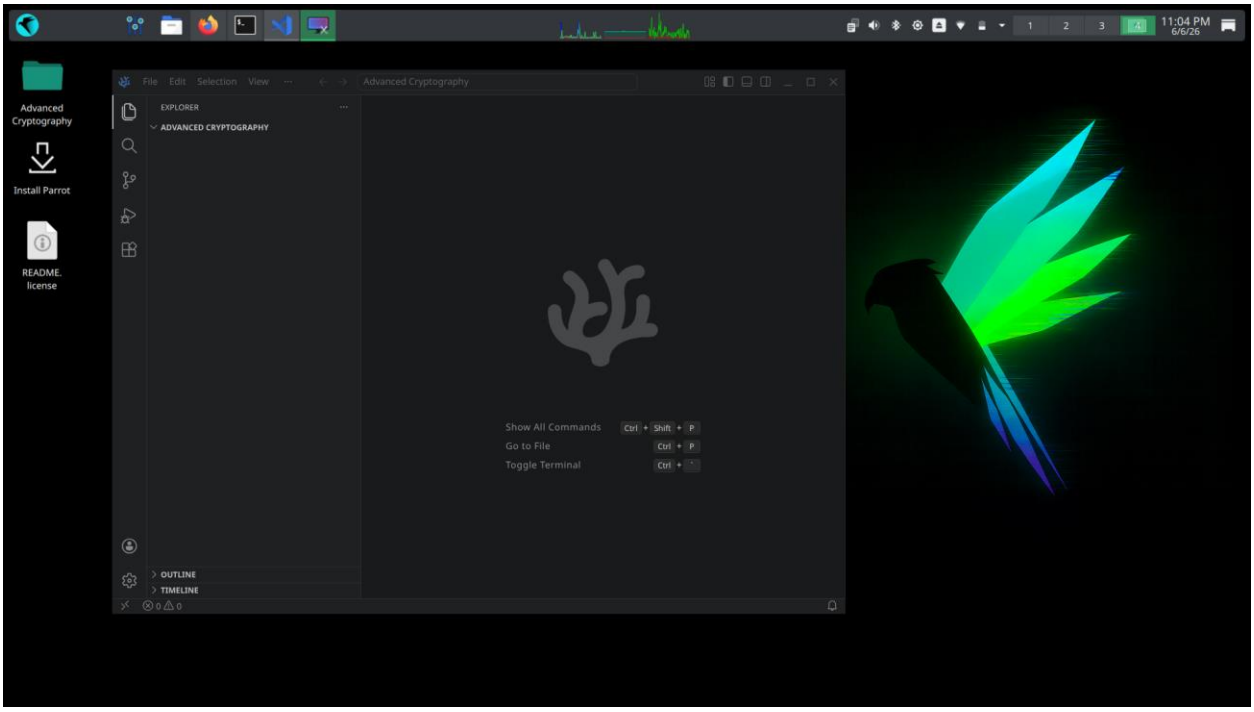


Fig 2: Installation of Cryptography Libraries

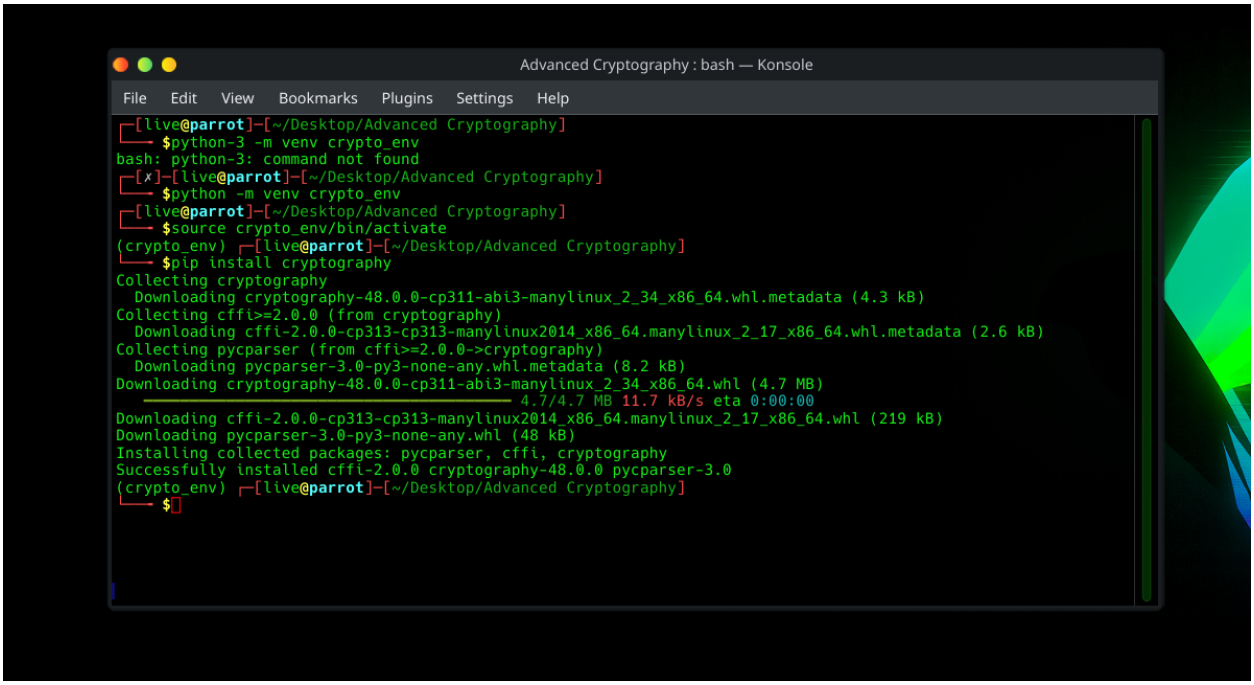


Fig 3: OpenSSL Installation Test

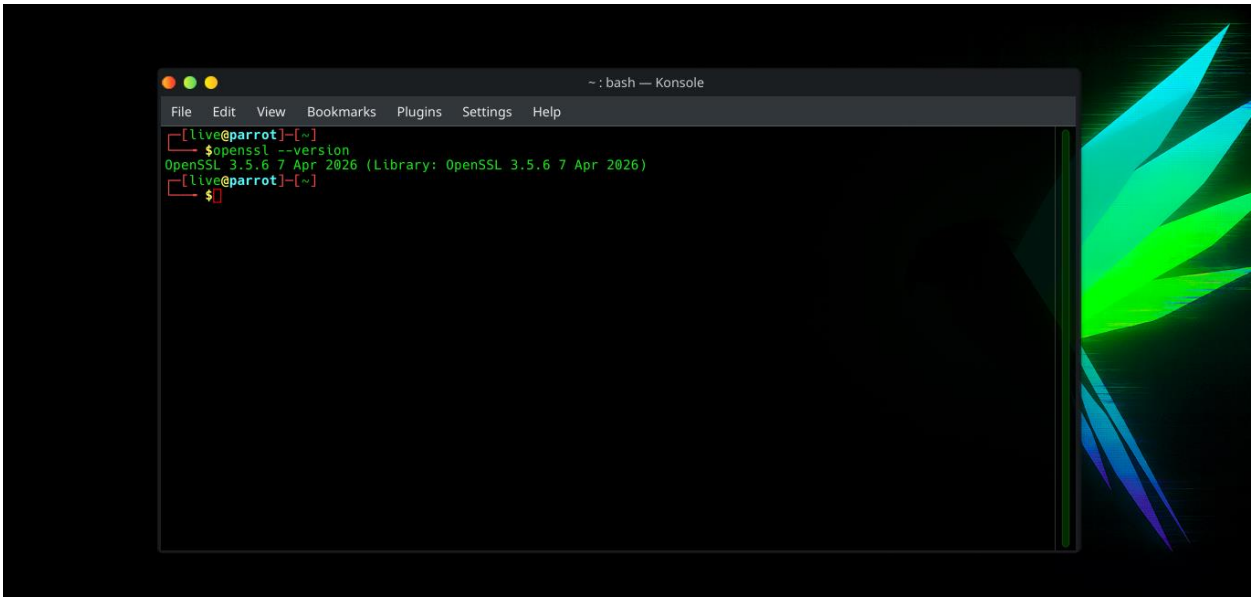


Fig 4: Local Encryption Script Execution

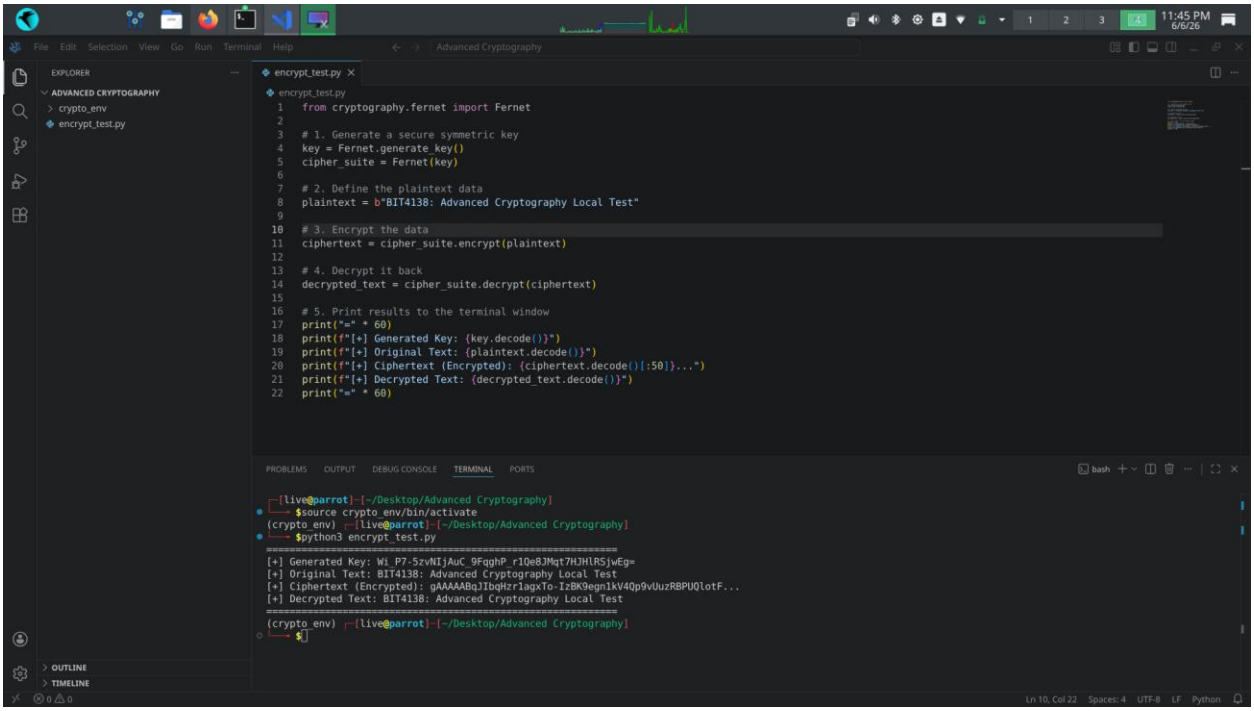
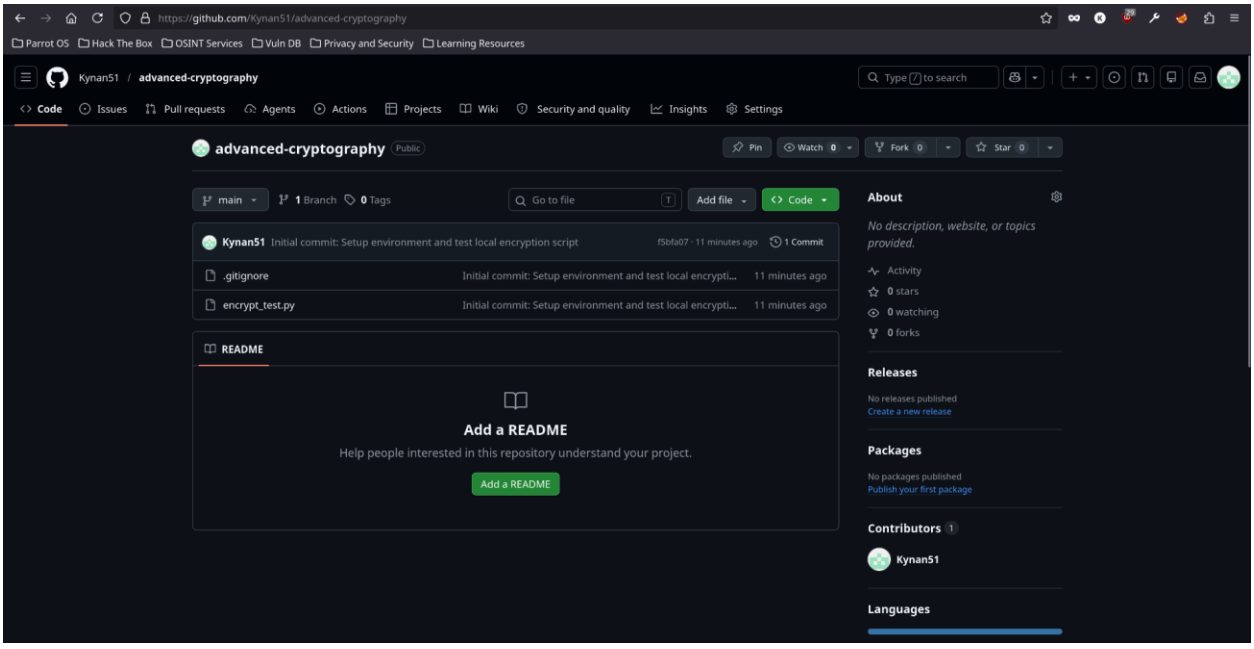


Fig 5: GitHub Repository Initialization



Student Reflection

Configured a portable development environment on a live persistent Parrot OS workstation. Verified the pre-installed OpenSSL architecture and established secure version control via Git. Fig 3 confirms successful installation and configuration of OpenSSL used for generating secure cryptographic keys and certificates.

Week 2: Classical Cryptography and Cipher Design

Implementation of Classical Encryption Algorithms

Fig 1: Caesar Cipher Implementation

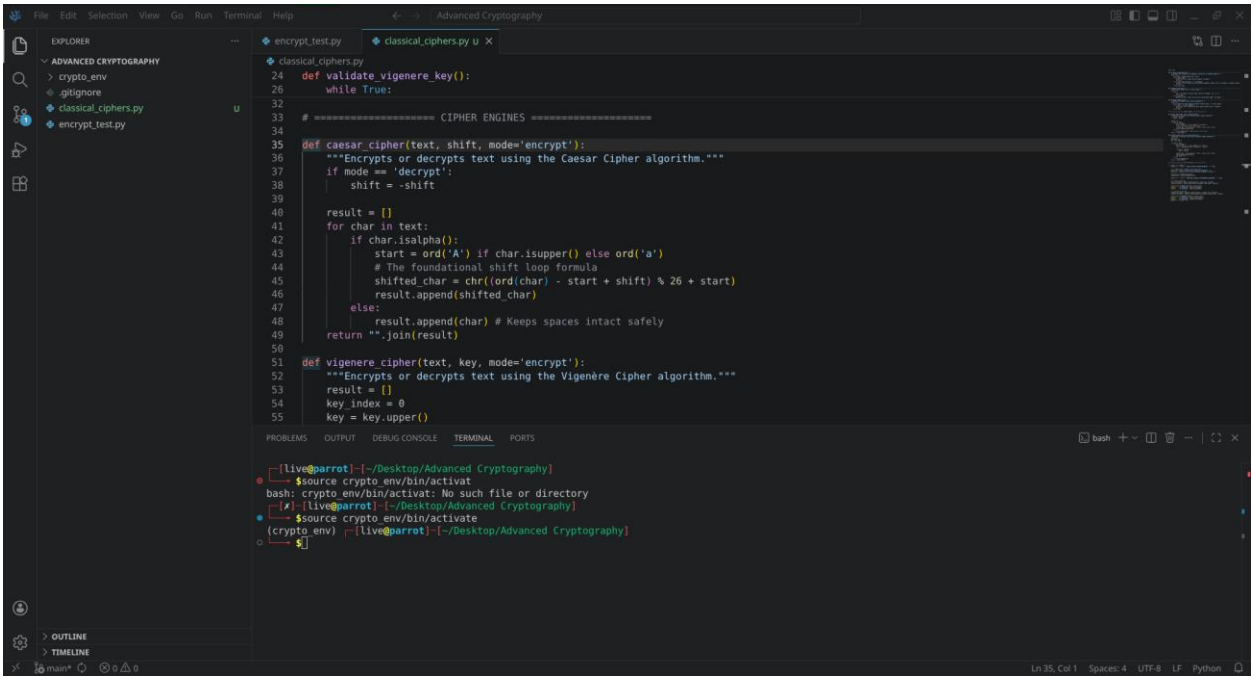


Fig 2: Vigenère Cipher Encryption Process

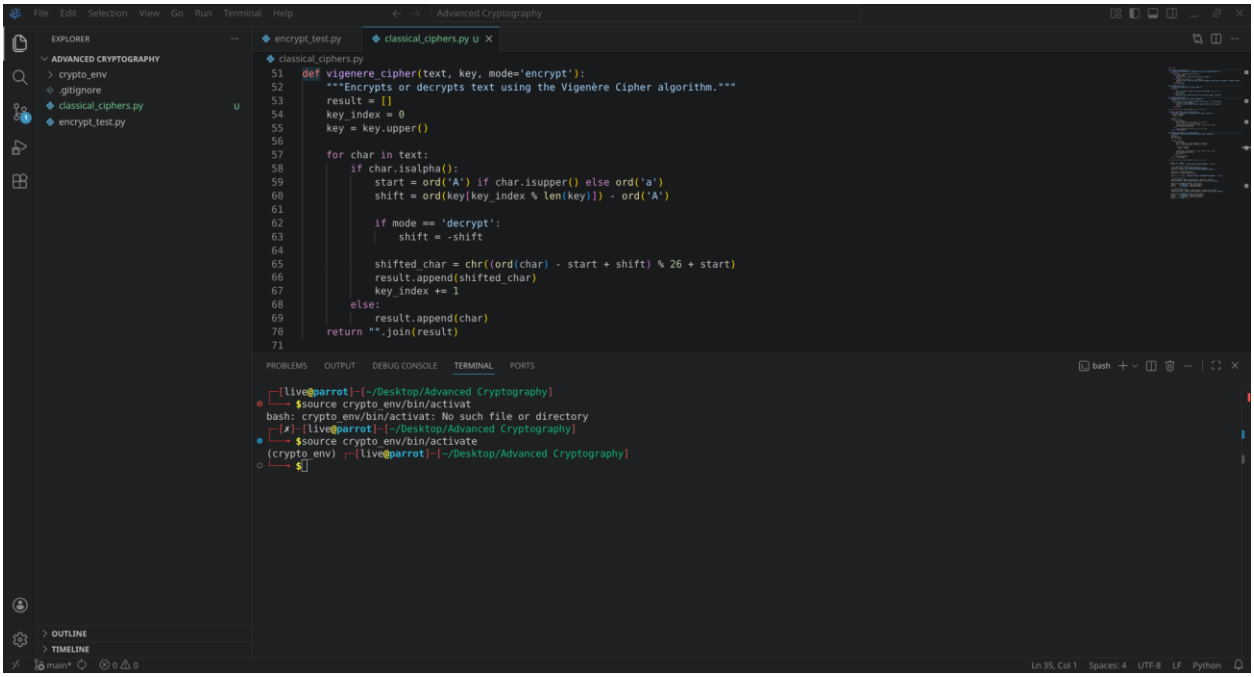


Fig 3: Encryption and Decryption Output

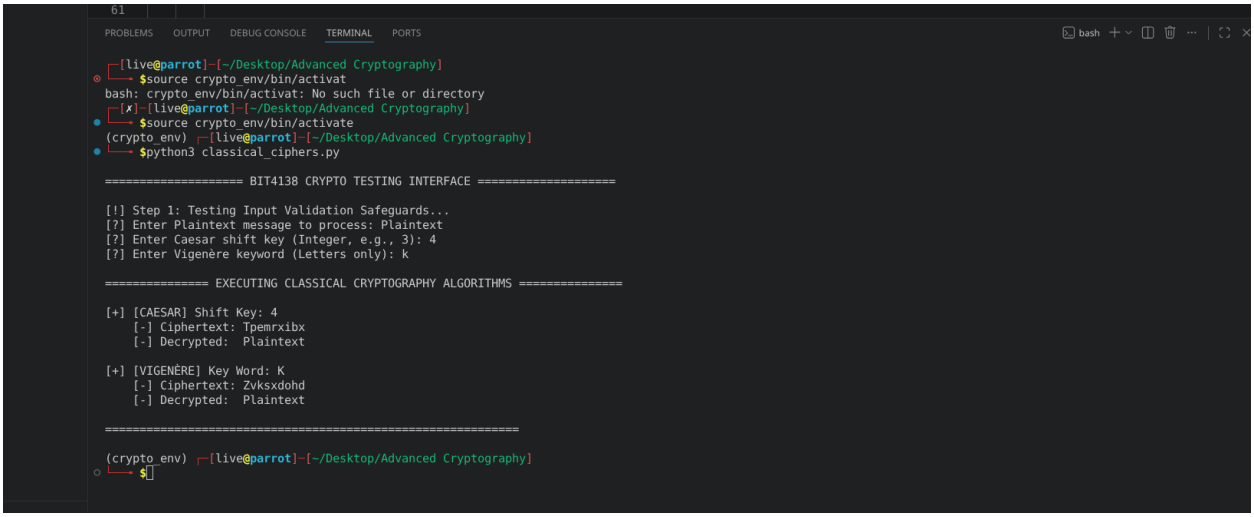


Fig 4: User Input Validation Interface

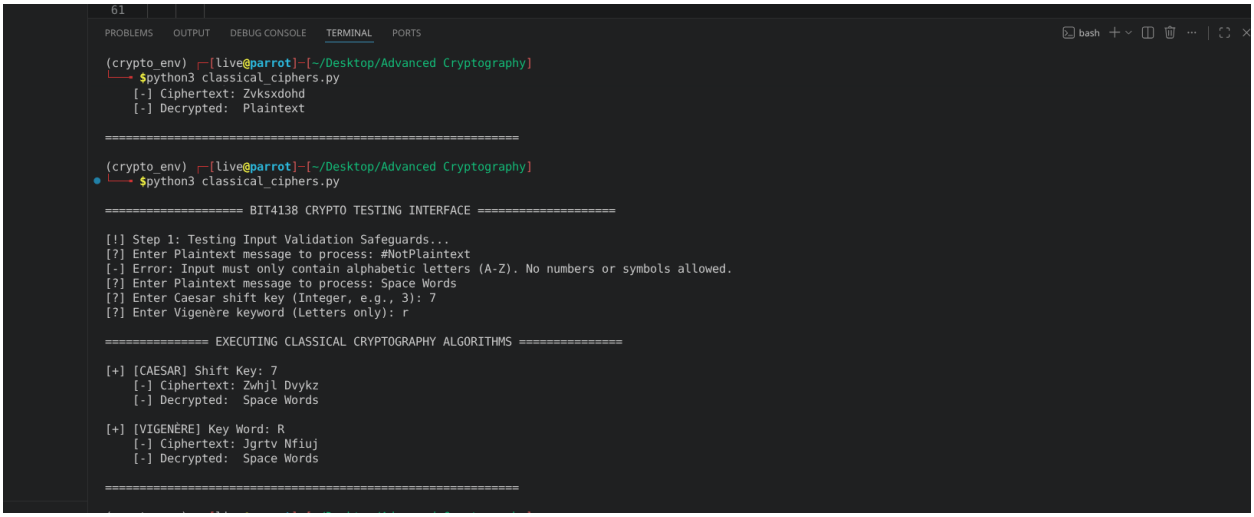
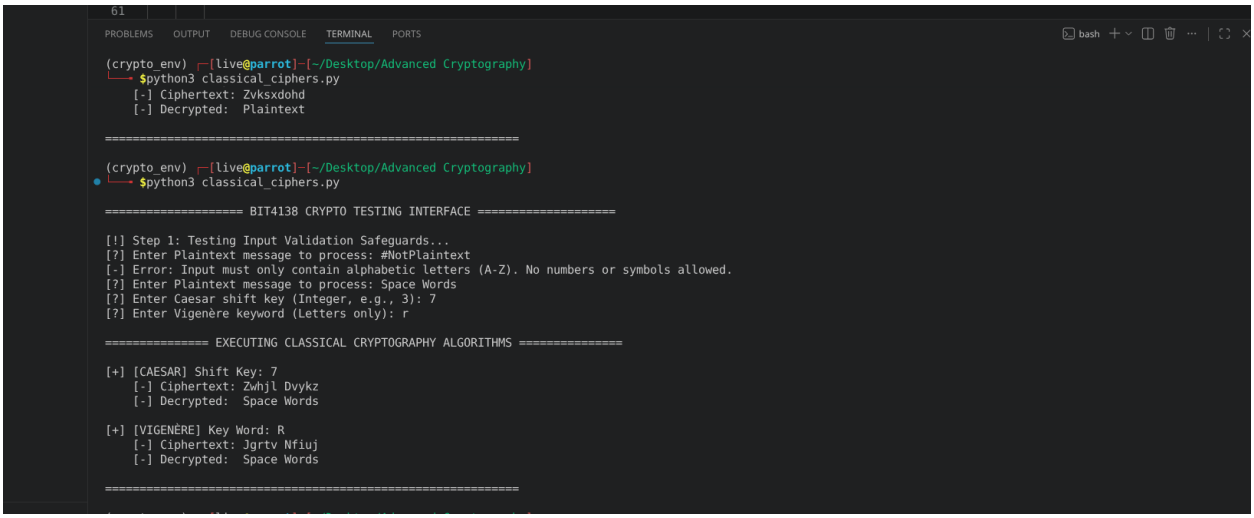


Fig 5: Cipher Testing Results



Student Reflection

Implemented functional Caesar and Vigenère polyalphabetic cipher architectures inside Parrot OS to analyze traditional data transformation rules. Developed input sanitization checks to handle structured data constraints defensively. The Vigenère implementation successfully demonstrated how using a variable alphabet rotation counter flattens pattern distributions to mitigate basic statistical frequency cryptanalysis vulnerabilities

Week 3: Stream Ciphers and Randomness Testing

Implementation of Stream Cipher Systems and Random Sequence Analysis

Fig 1: LFSR Generator Implementation

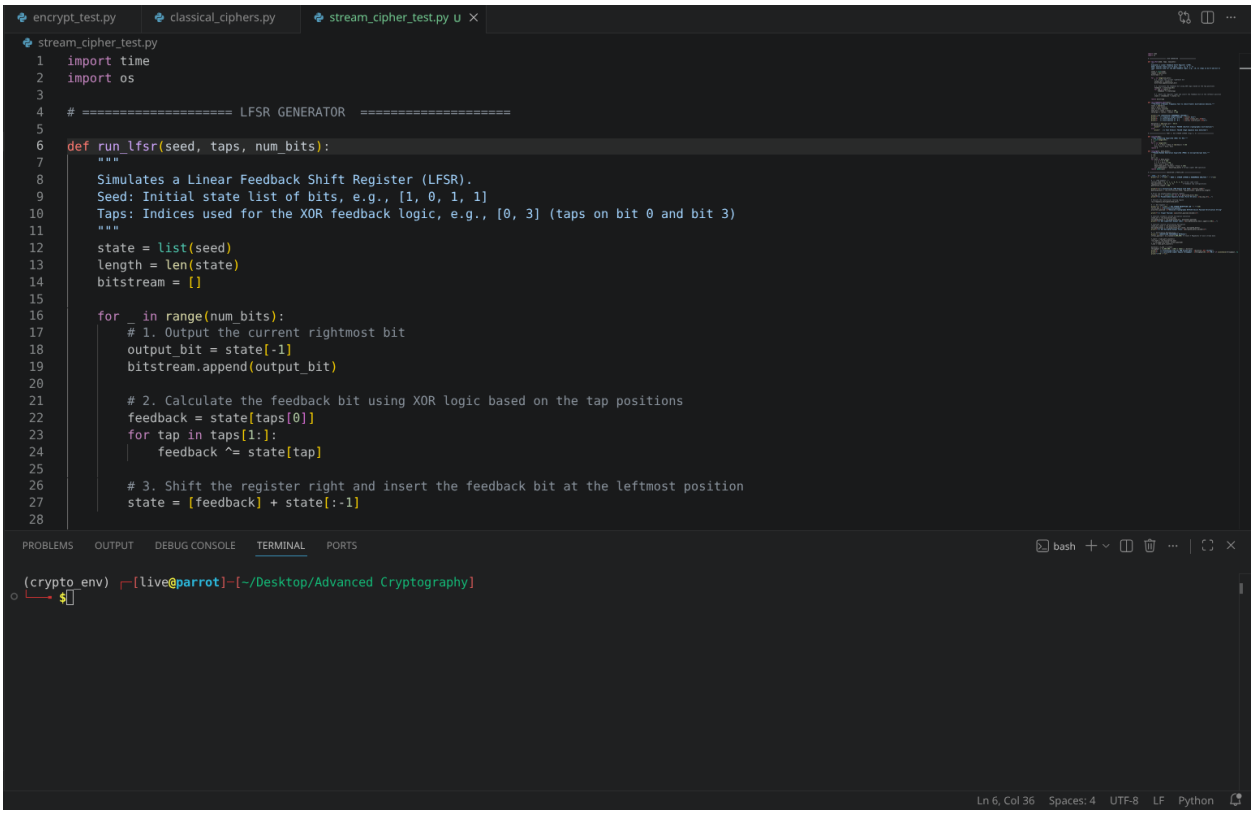


Fig 2: Pseudorandom Sequence Output

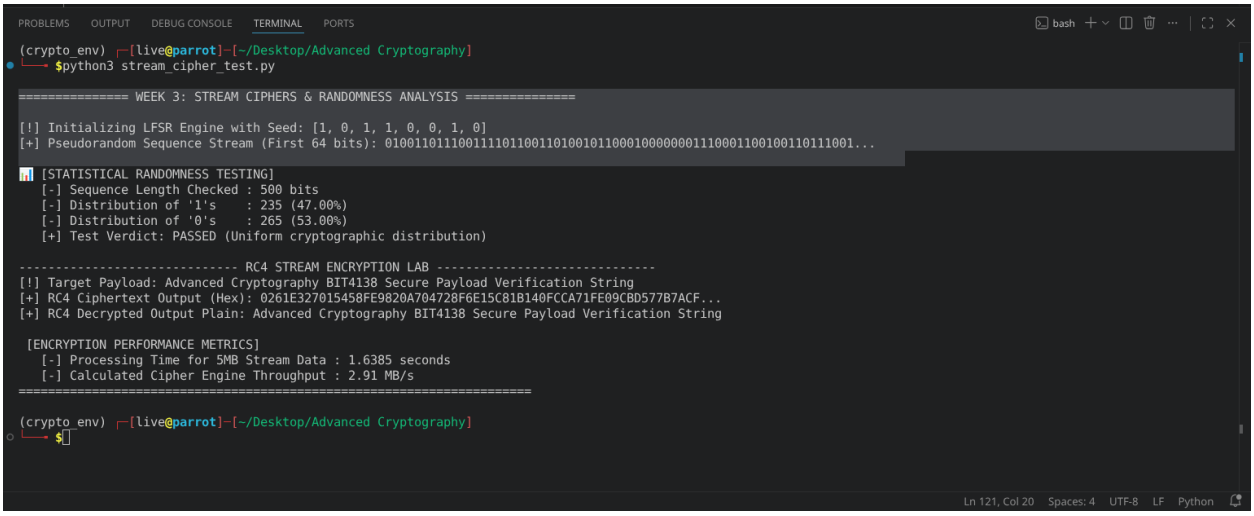


Fig 3: Statistical Randomness Testing

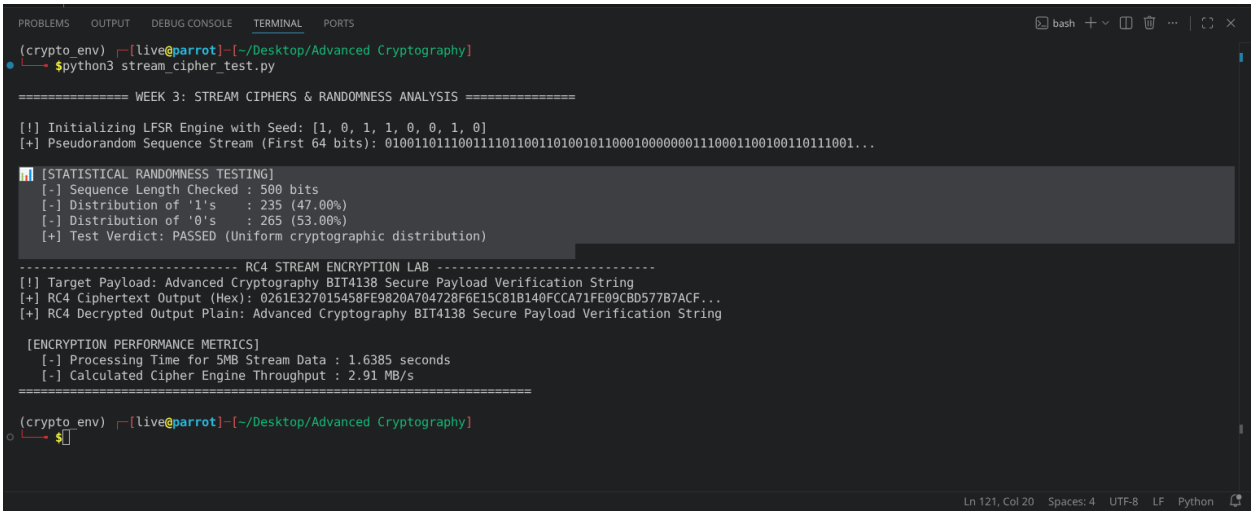


Fig 4: RC4 Stream Cipher Simulation

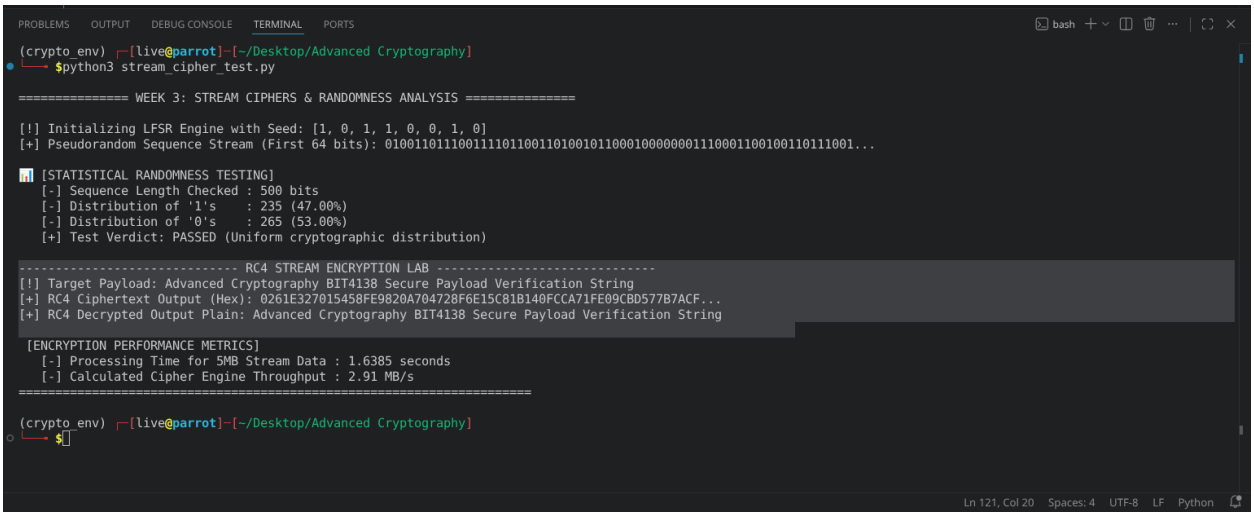
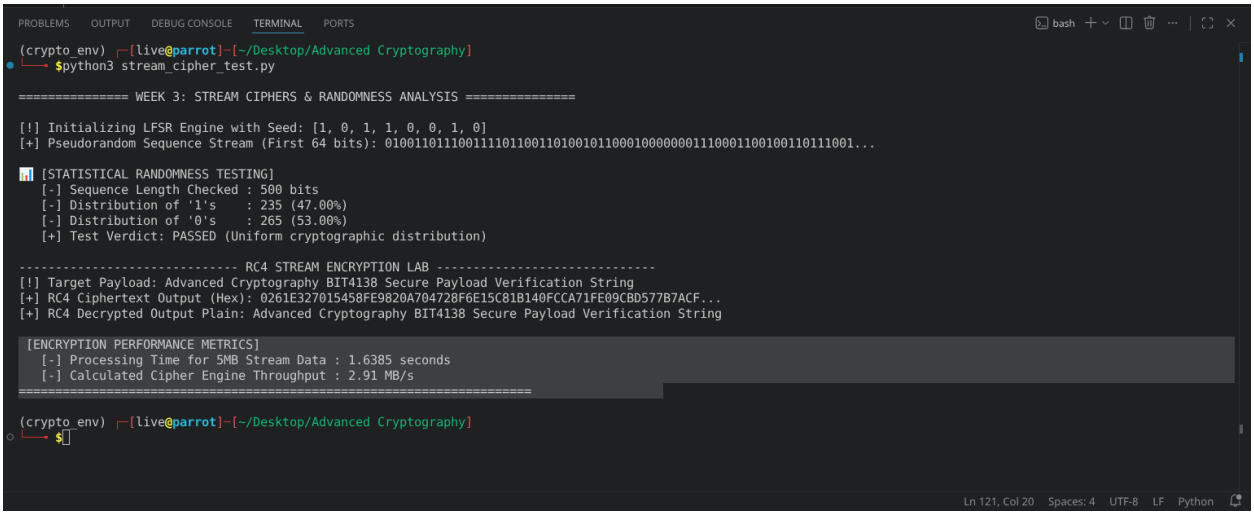


Fig 5: Encryption Performance Results



Student Reflection

Simulated an LFSR and RC4 engine to examine continuous byte-level encryption. Cryptographic randomness creates unpredictable key streams to defend against plaintext structural recognition. Executed monobit statistical testing to verify uniform distribution balance, confirming that high-speed throughput and low latency are successfully achieved without introducing mathematical sequence patterns.

Week 4: Block Cipher Design and AES

Advanced Encryption Standard (AES) and Block Cipher Operations

Fig 1: AES Encryption Script

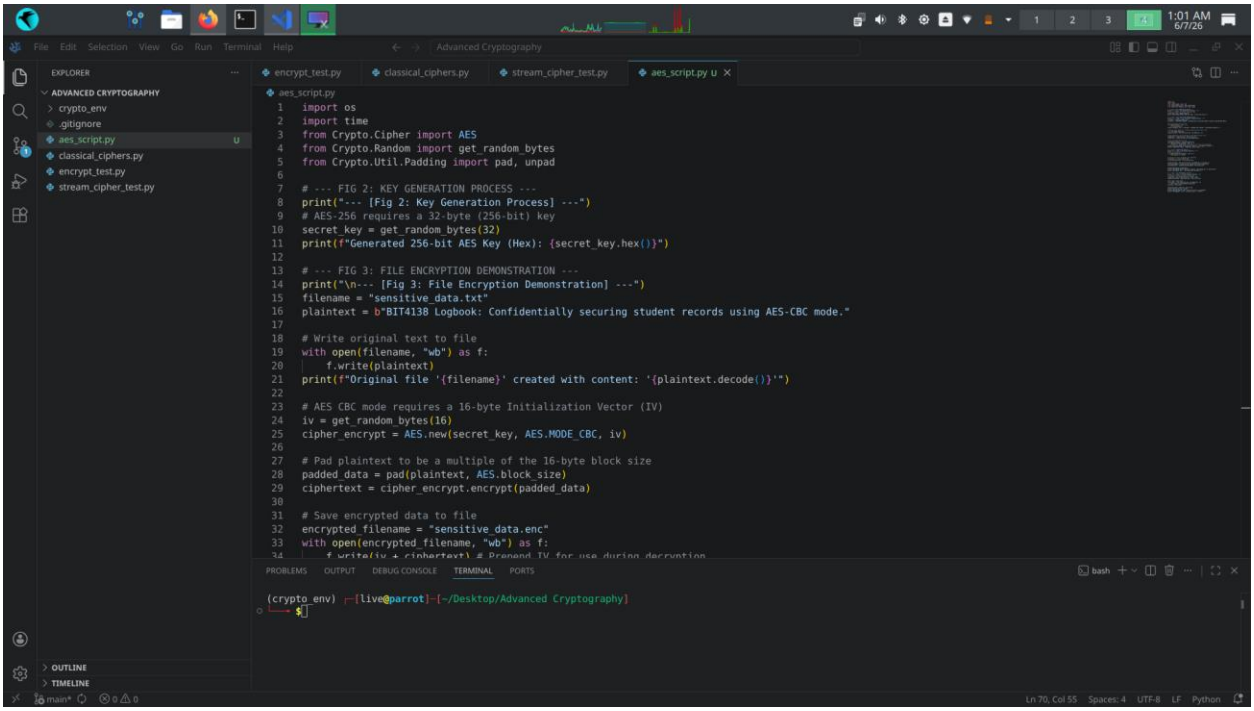


Fig 2: Key Generation Process

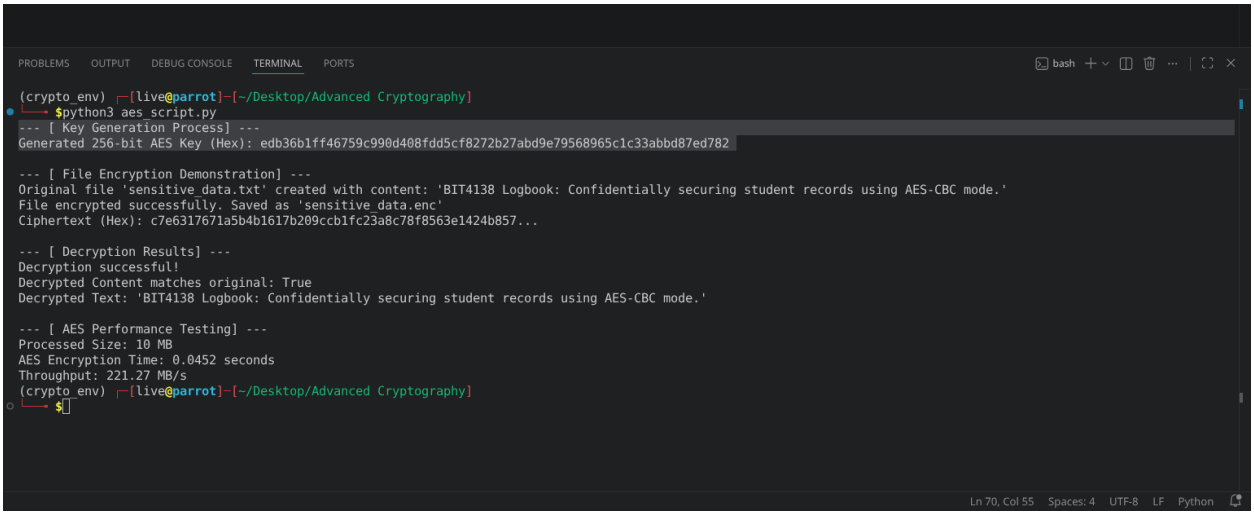


Fig 3: File Encryption Demonstration

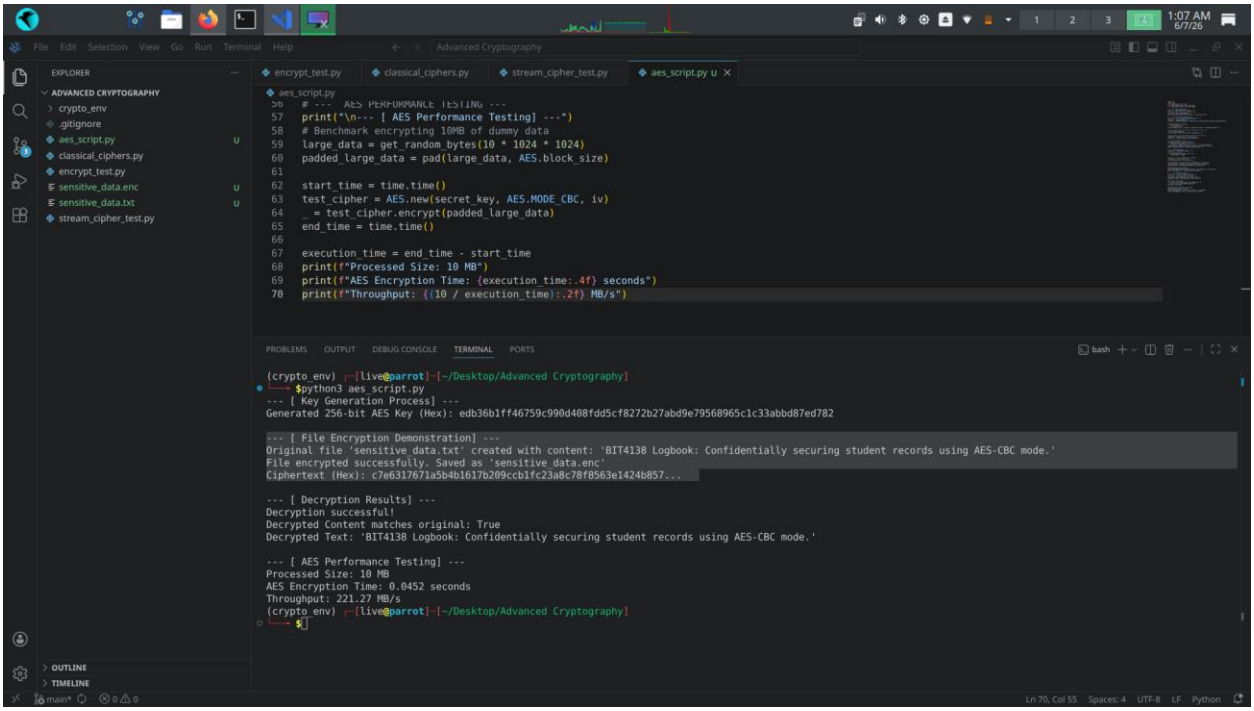


Fig 4: Decryption Results

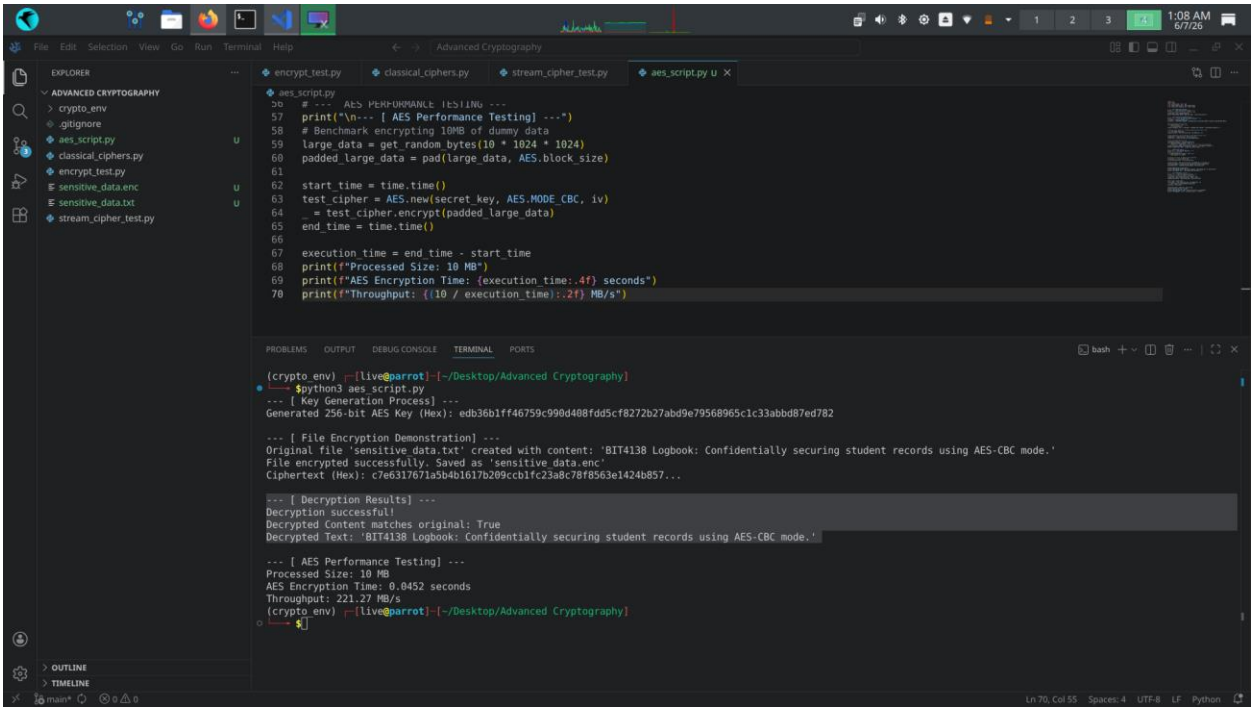
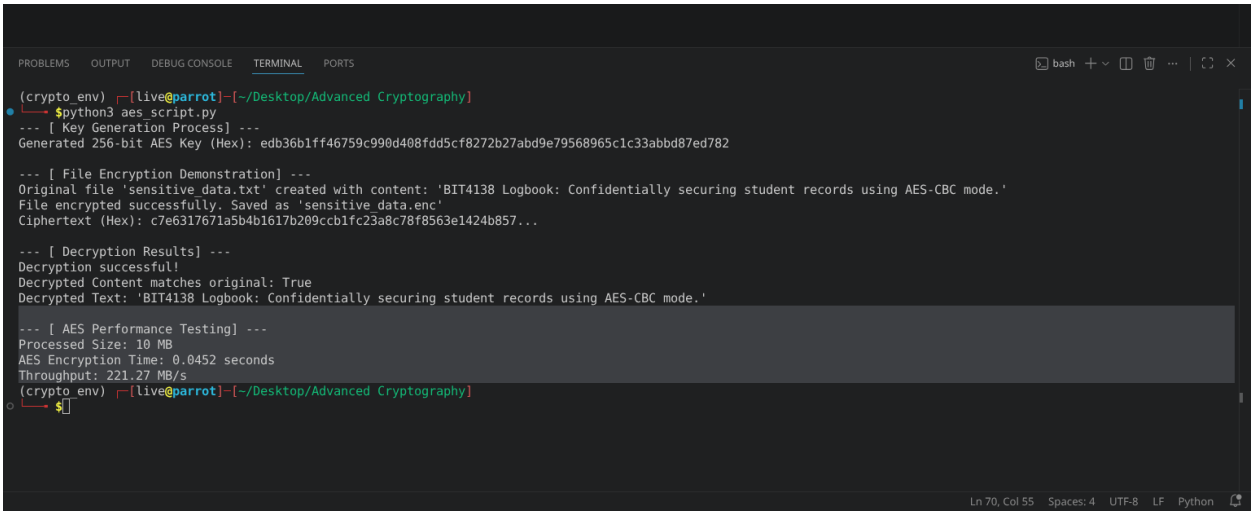


Fig 5: AES Performance Testing



Student Reflection

Implemented AES-256 in CBC mode, mastering its block-by-block diffusion workflow. Correctly padding data and securely handling the initialization vector (IV) was challenging. Secure key management proved vital, as compromised keys invalidate the entire encryption/decryption integrity. Performance testing highlighted AES's exceptional, hardware-optimized computational efficiency with large files.

Week 5: Public Key Cryptography (RSA)

Implementation of RSA Encryption and Key Management

Fig 1: RSA Key Pair Generation

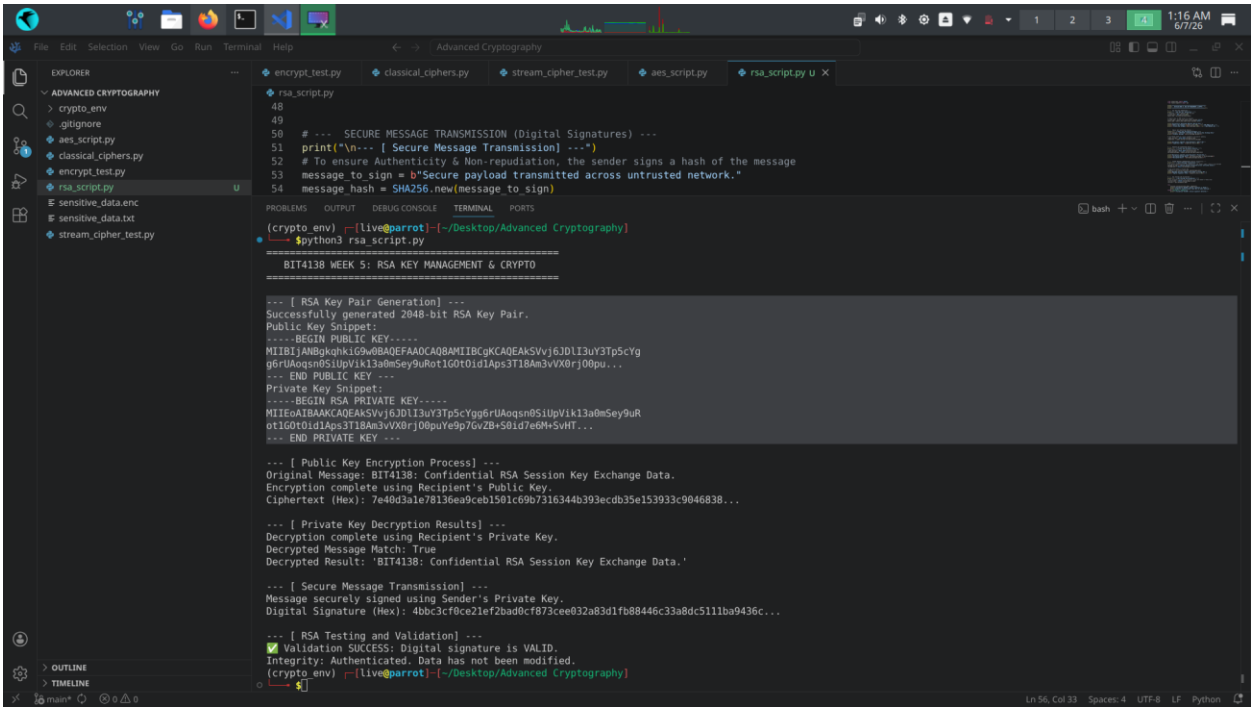


Fig 2: Public Key Encryption Process

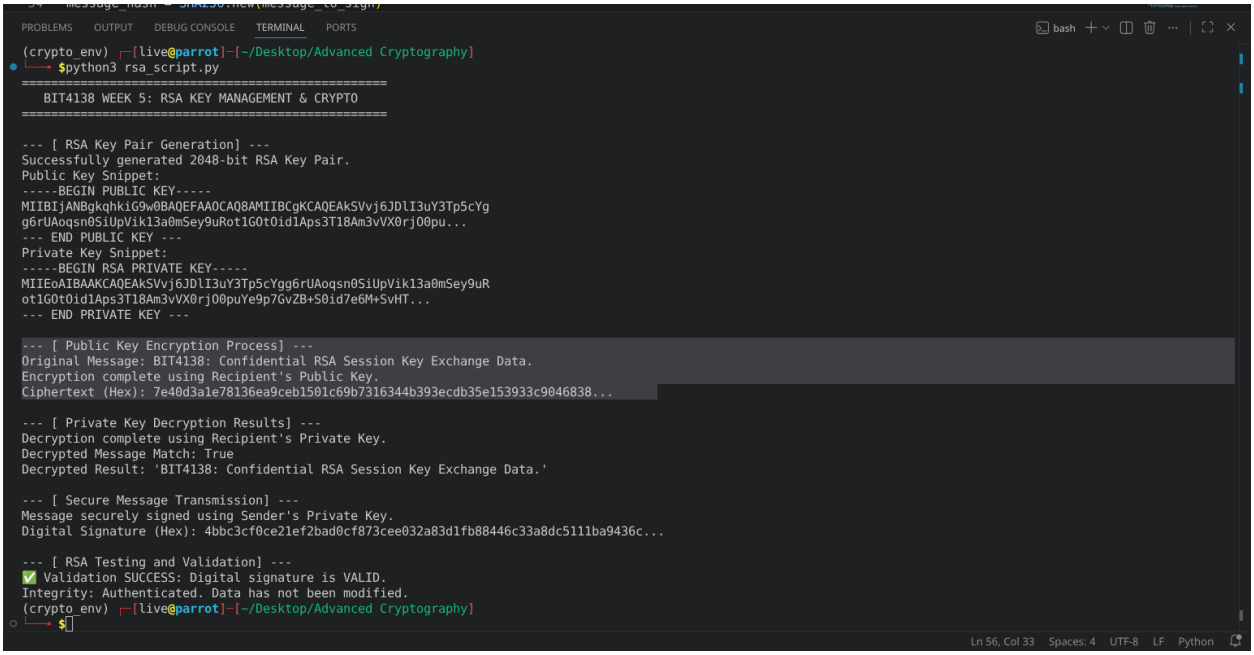


Fig 3: Private Key Decryption Results

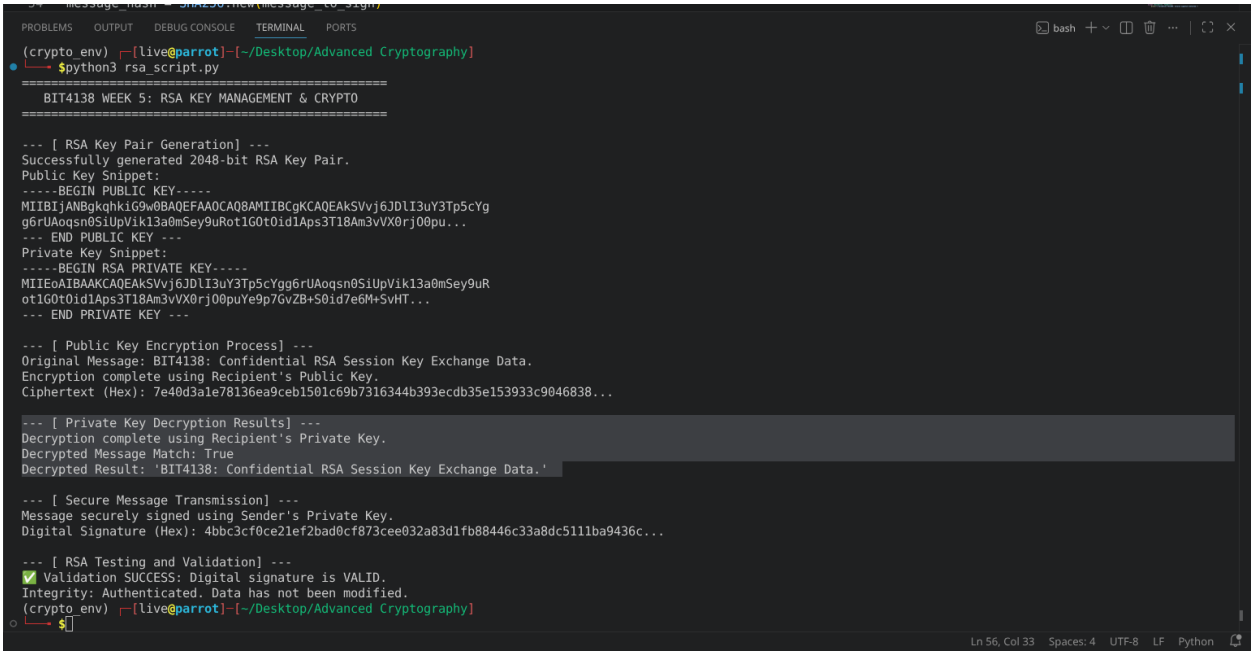


Fig 4: Secure Message Transmission

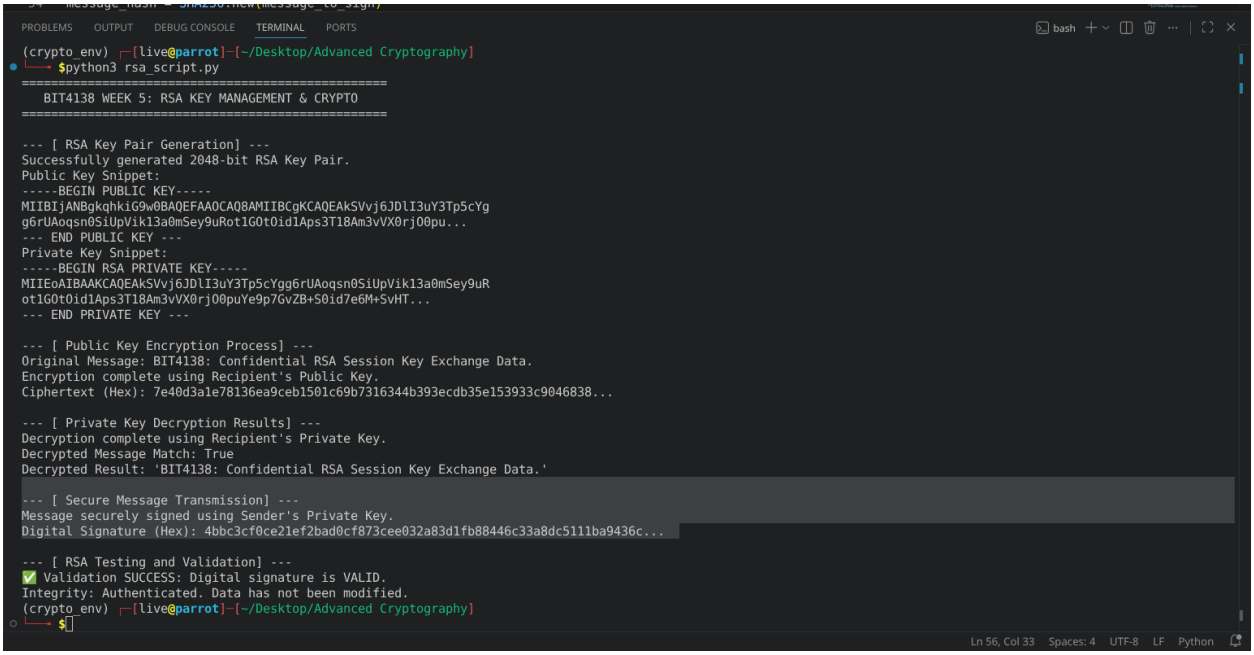
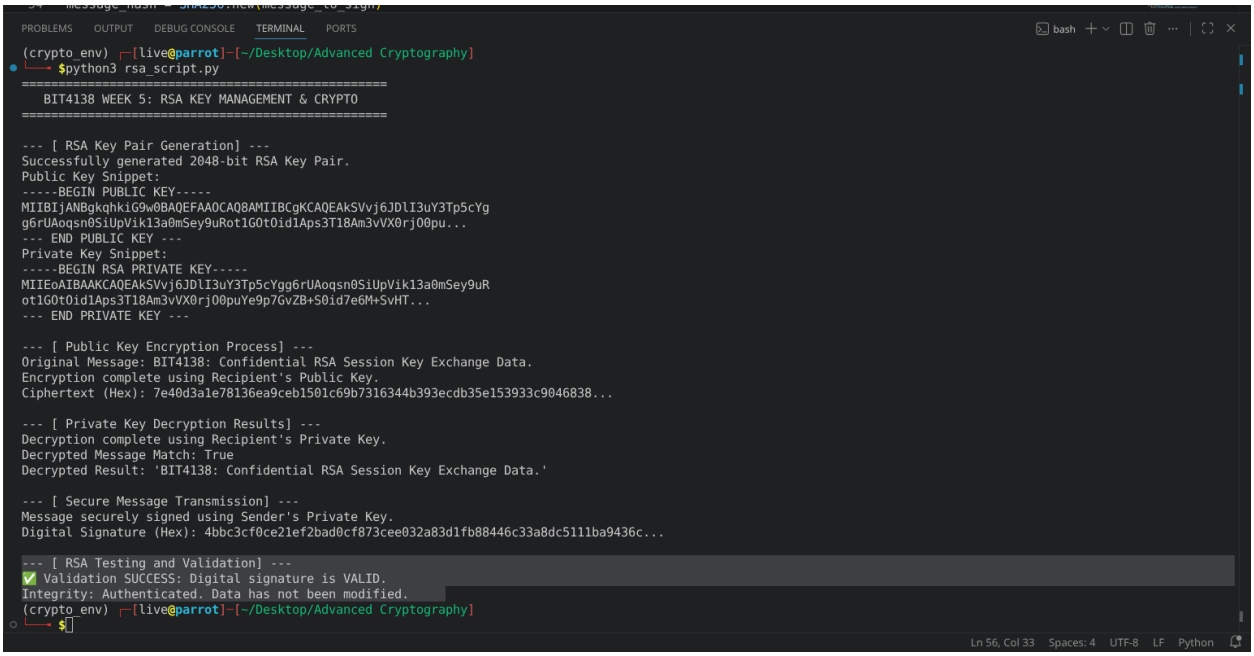


Fig 5: RSA Testing and Validation



Student Reflection

Configured an asymmetric 2048-bit RSA key pair to evaluate public-key infrastructure. I leveraged the public key for data encryption and digital signature verification, while reserving the private key for decryption. Managing mathematical overhead and ensuring

signature validation padding configurations were challenging, yet the process perfectly underscored core message integrity mechanisms.

Week 6: Hashing and Password Security

Implementation of Secure Hash Functions and Authentication

Fig 1: SHA-256 Hash Generation

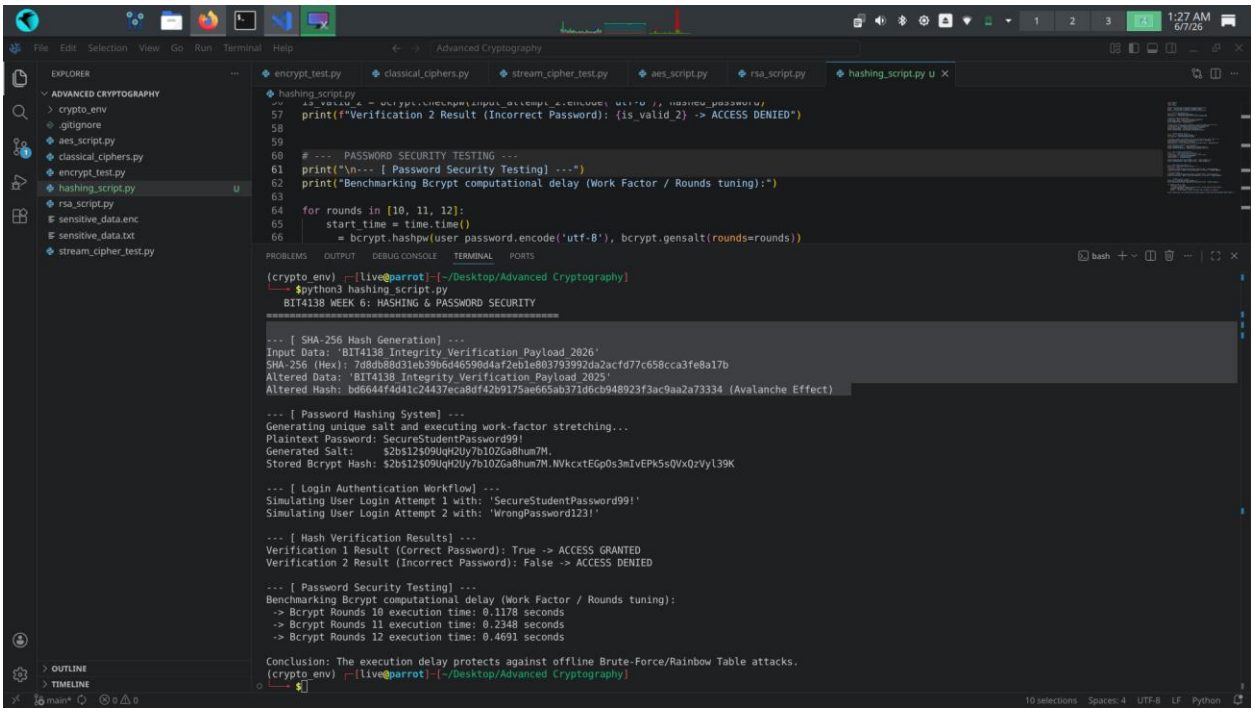


Fig 2: Password Hashing System

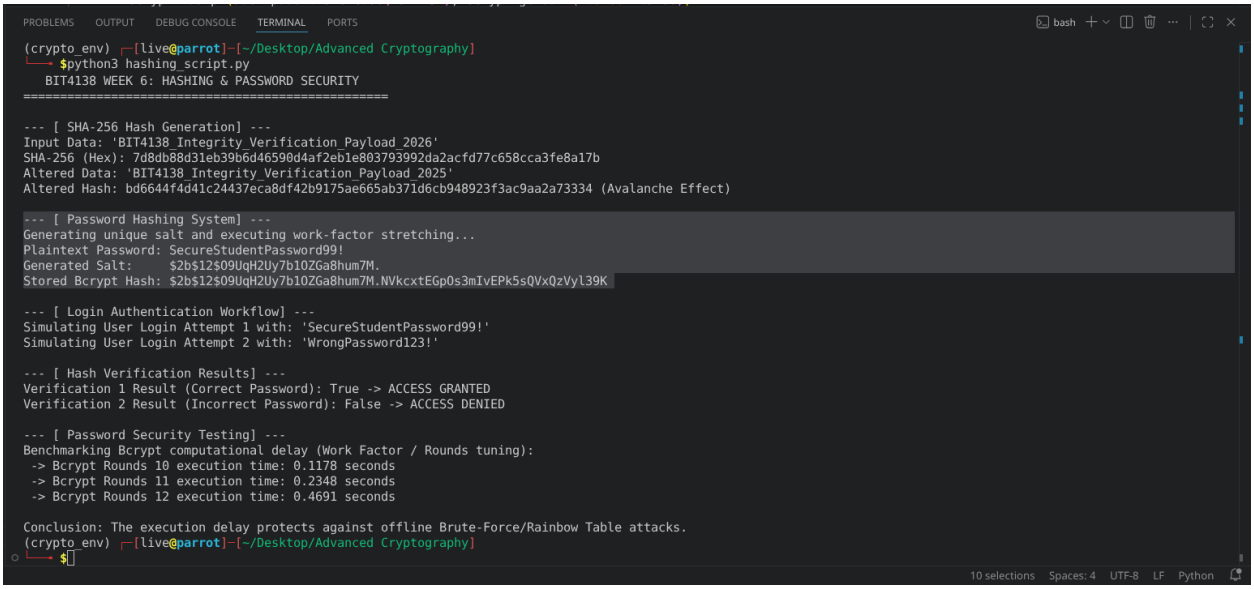


Fig 3: Login Authentication Workflow

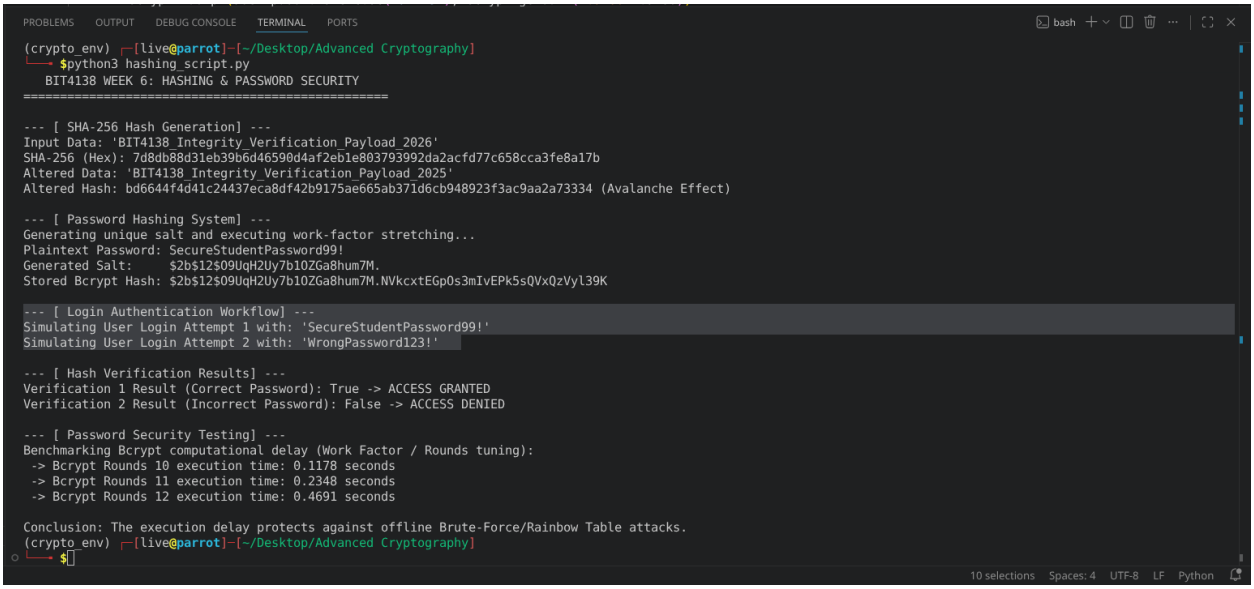


Fig 4: Hash Verification Results

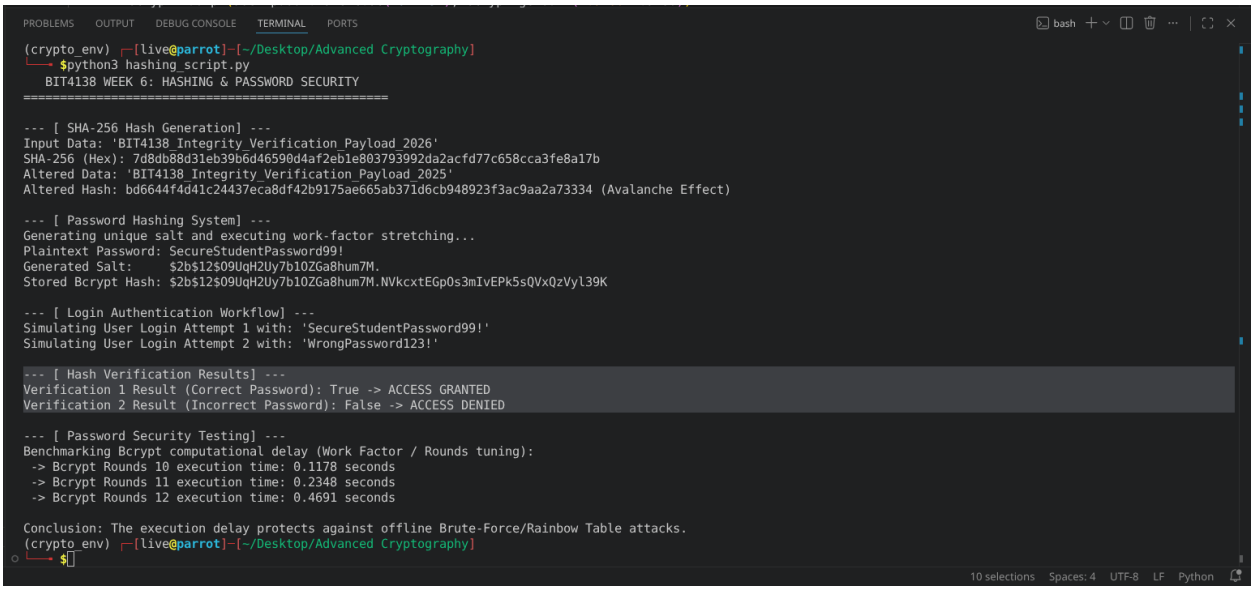
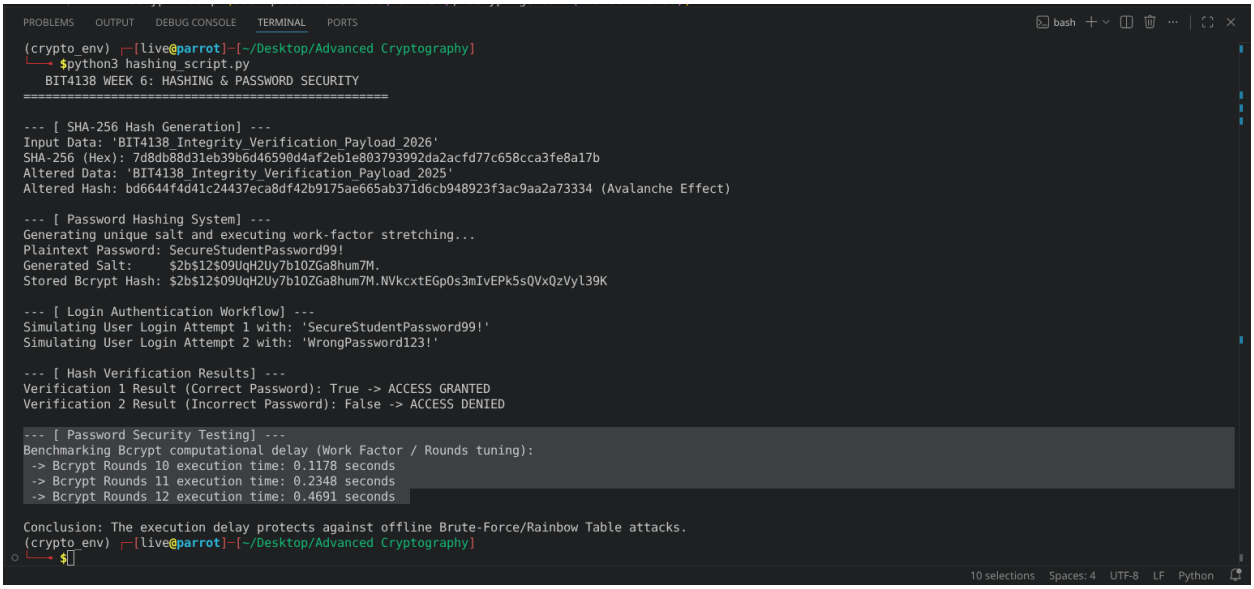


Fig 5: Password Security Testing



Creative Task: Password Strength Analyzer and Leak Simulator

Fig 1: Password Entropy and Failure Mode

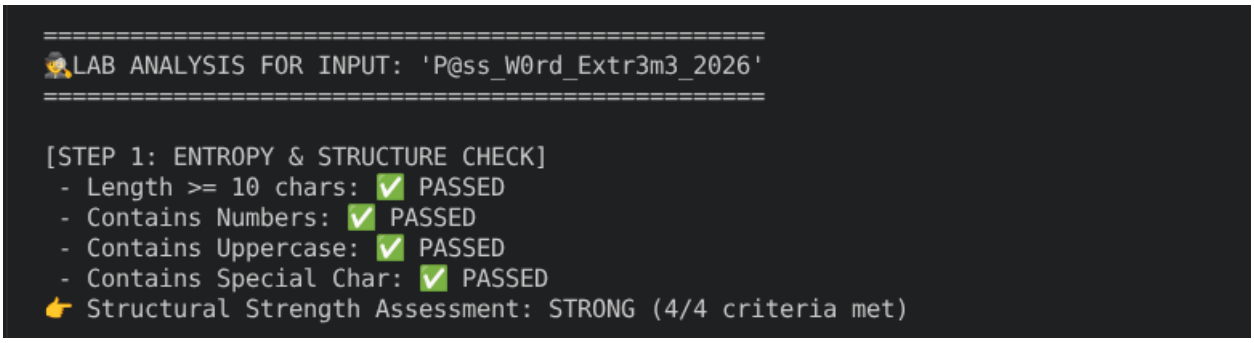


Fig 2: Simulated Breach / Rainbow Table Match

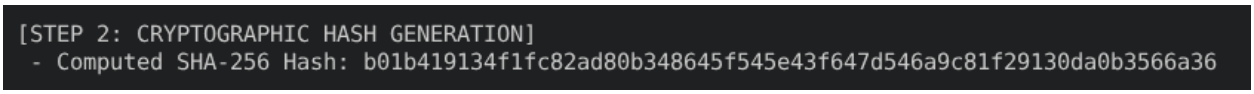
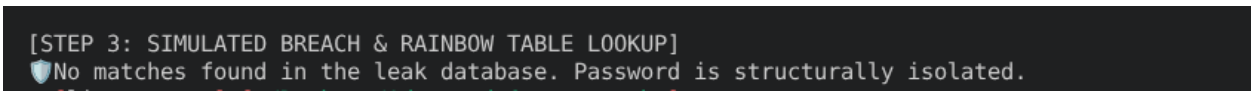


Fig 3: Secure Cryptographic Isolation



Student Reflection

Evaluated SHA-256 data integrity tracking and implemented a resilient password storage system using bcrypt. I explored how salting neutralizes precomputed rainbow table vector

threats while algorithmic stretching stops brute-force tools. Mitigating type errors during byte-string formatting proved challenging, but the metrics validated how computational latency properly defends user credentials.

Week 7: Digital Signatures and Certificates

Digital Signature Implementation and Certificate Management

Fig 1: Digital Signature Generation

```
┌──[live@parrot]─[~/Desktop/Advanced Cryptography]
└─$python3 signature_system.py
---  SIGNATURE GENERATION ---
Document Message: Academic Logbook Data: Integrity Verified.
Generated PSS Signature (Hex): c640736af9c82cba8618ce16f96ed4371e5de37b746744c889ec9a7c870a7b26...
```

Fig 2: Signature Verification Process

```
# 4. Signature Verification Process
print("\n--- SIGNATURE VERIFICATION & VALIDATION ---")
try:
    public_key.verify(
        signature,
        document_data,
        padding.PSS(mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.MAX_LENGTH),
        hashes.SHA256()
    )
    print("[SUCCESS] Signature matches document. Integrity & Authenticity confirmed!")
except Exception:
    print("[FAILURE] Signature validation failed.")

```

Fig 3: Certificate Creation Using OpenSSL

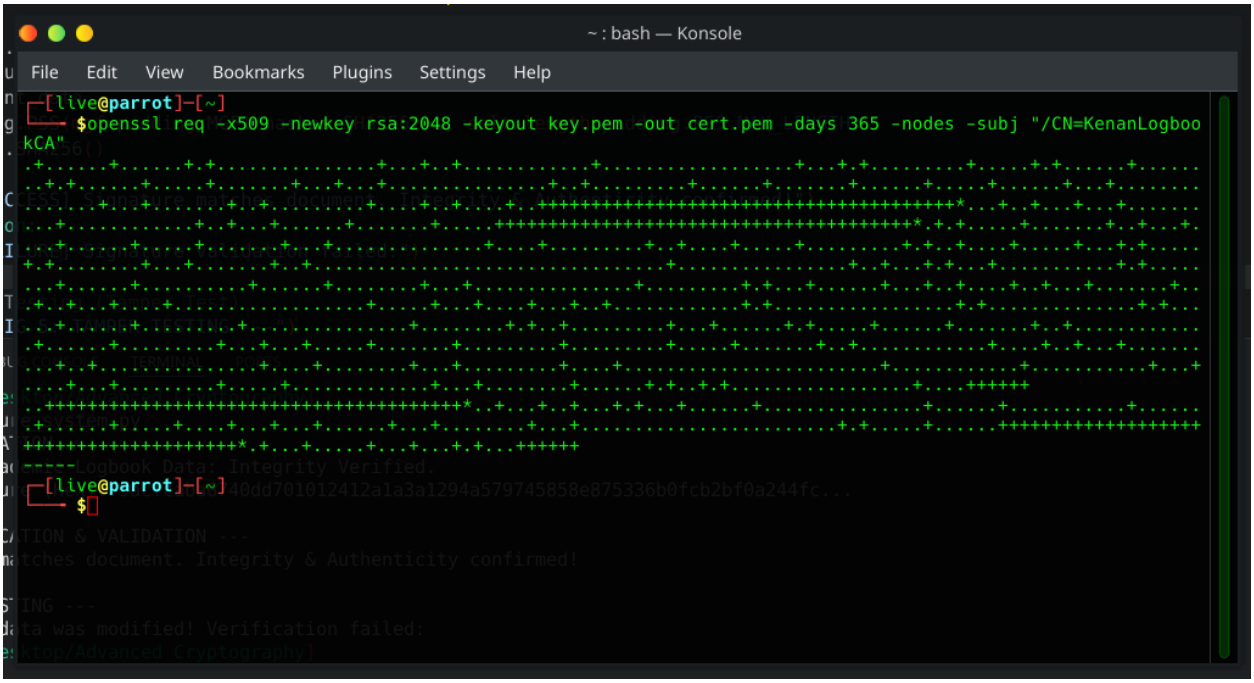


Fig 4: Secure Document Validation

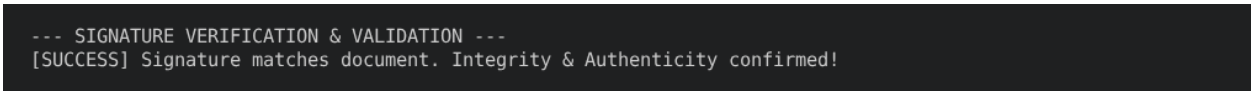
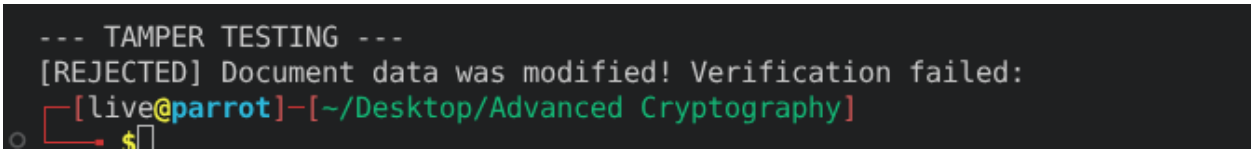


Fig 5: Certificate Testing Results



Creative Task: Software Update Integrity Guard

Fig 1: Cryptographic Packaging and Core Signing Execution

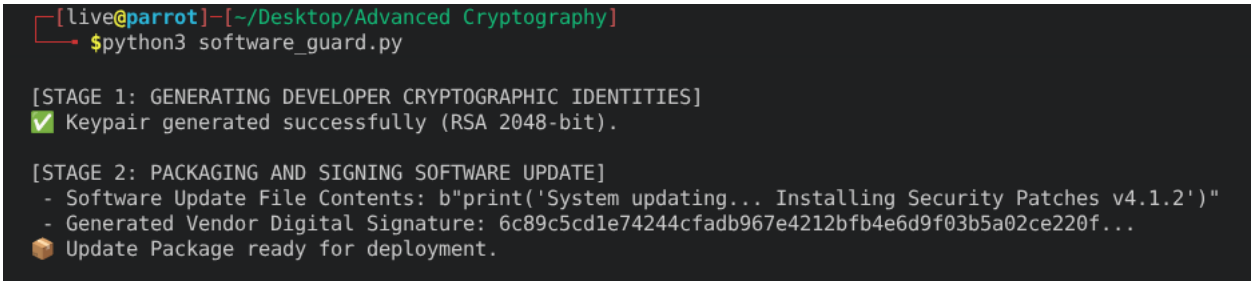


Fig 2: Client Verification Success & Deployment Authorization

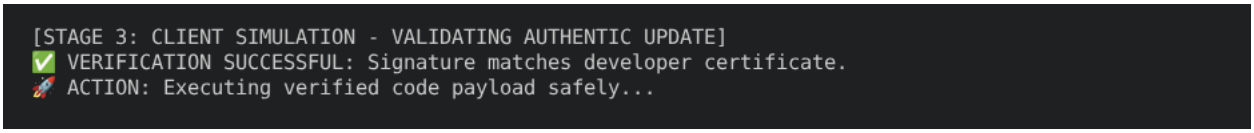
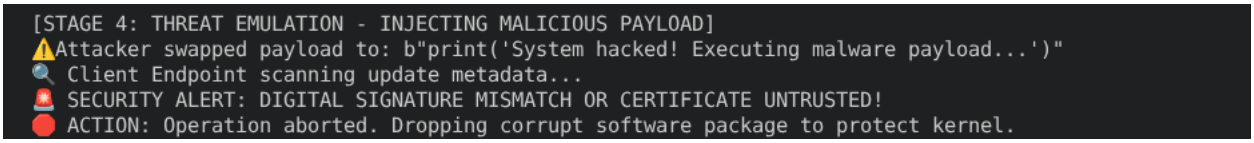


Fig 3: Malicious Interception and Tamper Defiance



Student Reflection

Digital signatures guarantee non-repudiation and data integrity. The authentication workflow binds a public key to an identity via certificates. Validation checks validity periods and cryptographic signatures against trusted roots. Real-world applications include secure e-commerce, software distribution signing, code signing, and securing HTTPS communications using SSL/TLS infrastructure globally.

Week 8: Public Key Infrastructure

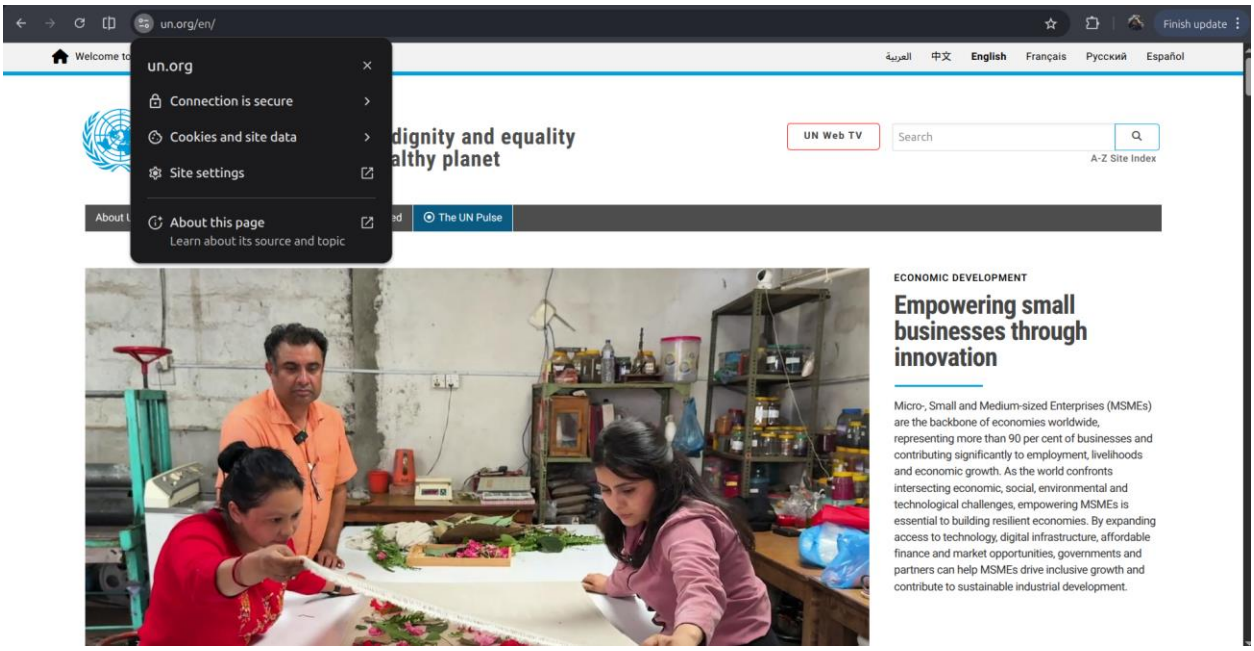
Part 1: PKI and Digital Certificates

Class Activity 1: PKI Component Identification

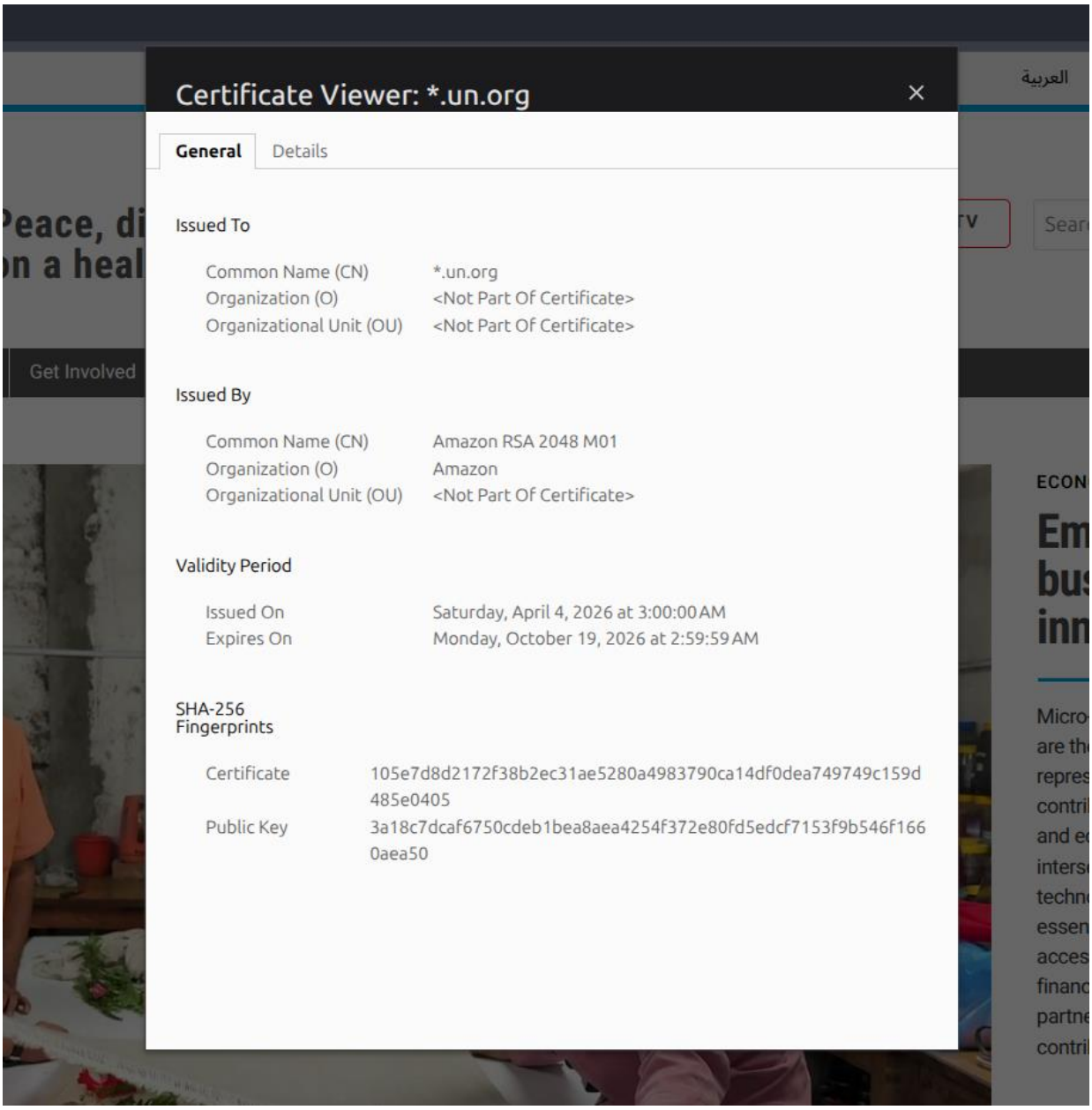
a) Certificate Authority (CA): A trusted organization that verifies identities, issues digital certificates, and manages their revocation.
b) Registration Authority (RA): An intermediary that verifies user identity and approves certificate requests before sending them to the CA.
c) Digital Certificate: An electronic document binding an entity's identity to its public key.
d) End User: Individuals, organizations, or systems (like web browsers) that use certificates to establish trust.
e) Certificate Repository: A database where issued certificates are stored and accessed.

Class Activity 2: Certificate Analysis

Selected Website: United Nations Website

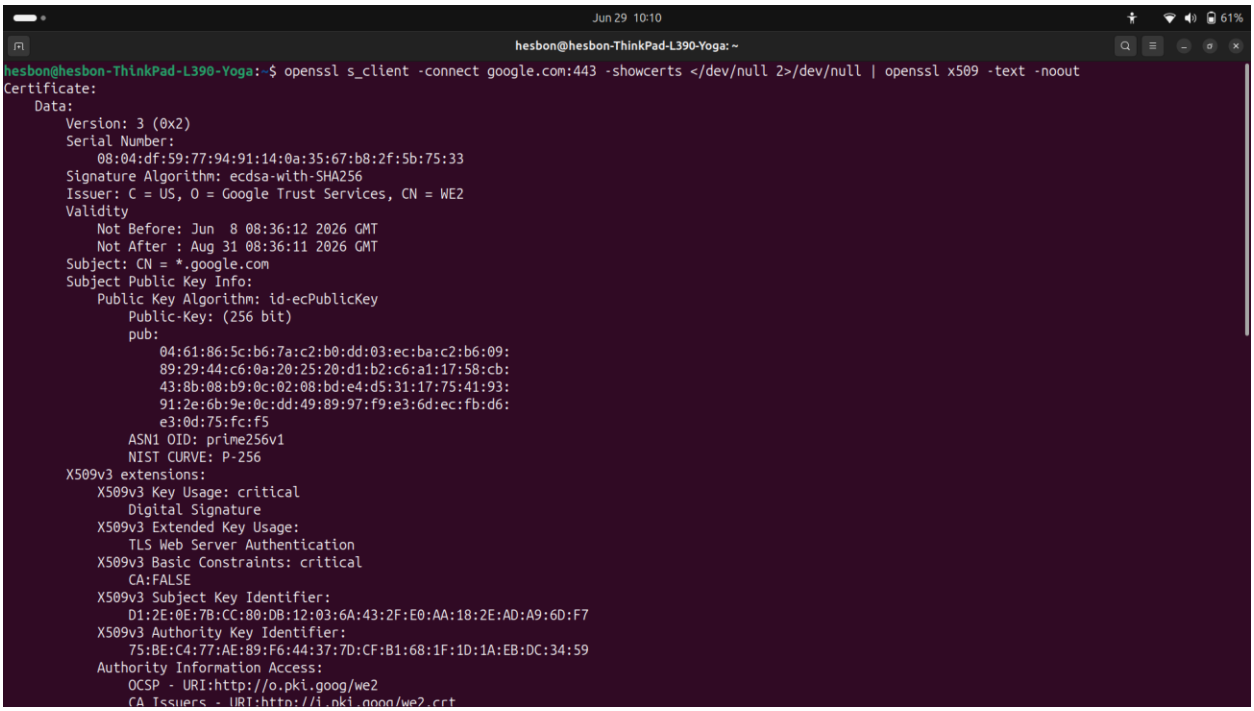


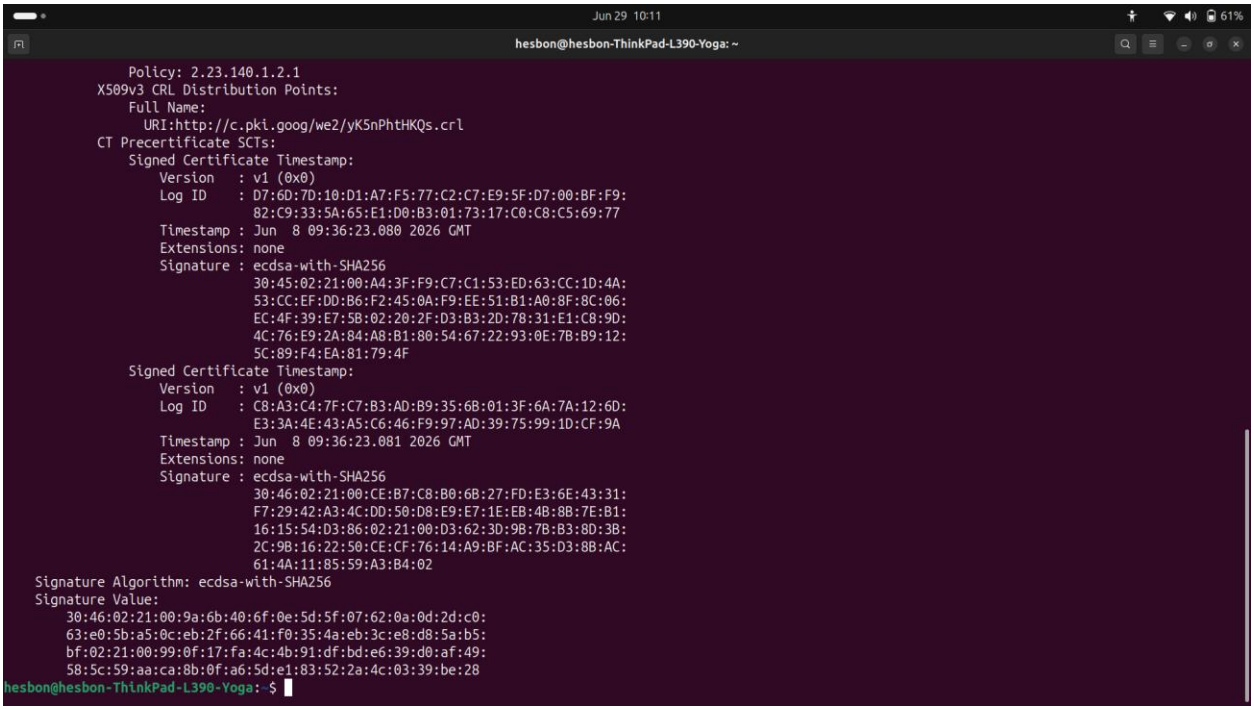
Certificate Information



OpenSSL Certificate Inspection

Google.com Certificate Inspection





Digital Certificate Security Incident Report

The DigiNotar breach occurred when Iranian hackers compromised the CA, issuing over 500 fraudulent certificates, including one for Google. This allowed man-in-the-middle attacks spying on thousands of users. Consequently, browsers untrusted DigiNotar, forcing the company into bankruptcy. This underscored the need for certificate pinning and automated revocation methods.

Part 2: Asymmetric Ciphers and Diffie-Hellman

Class Exercise 1: Understanding Asymmetric Cryptography

a) Public Key: A publicly distributed key used to encrypt data or verify digital signatures.

b) Private Key: A strictly confidential key kept by the owner to decrypt data or generate digital signatures.

c) Symmetric Encryption: A system using a single shared key for both encryption and decryption.

d) Asymmetric Encryption: A system using a mathematically linked public-private key pair.

Class Exercise 2: Complexity and Cryptography

Computationally hard problems ensure that while encryption and decryption are fast for authorized parties, reversing the process without the key is computationally infeasible for attackers. Three examples include:

1. Discrete Logarithm Problem (used in Diffie-Hellman).
2. Integer Factorization (used in RSA).
3. Elliptic Curve Discrete Logarithm Problem (used in ECC).

Practical Task: Implementing Diffie-Hellman Key Exchange

Assignment 2: Diffie-Hellman in Modern Internet Security Report

The Diffie-Hellman protocol establishes shared secrets over insecure channels using the mathematically intractable Discrete Logarithm Problem. Essential to TLS/SSL handshakes, it secures HTTPS and VPN connections. Because it lacks built-in authentication, it is paired with digital signatures to prevent devastating Man-in-the-Middle exploits.

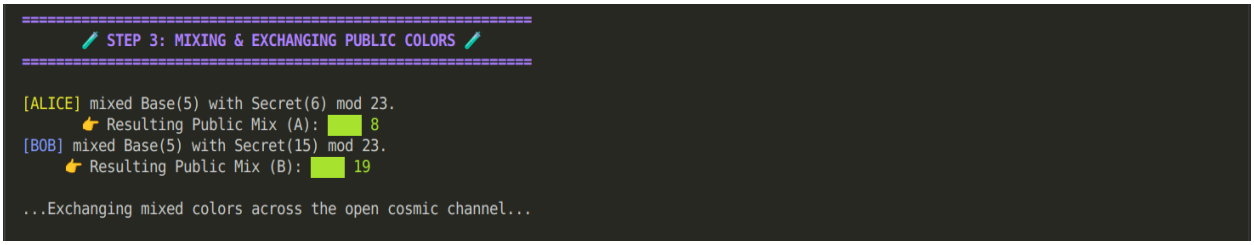
Creativity Assignment

A python script that models the **Diffie-Hellman Paint Analogy** visually in your terminal using colored text blocks.

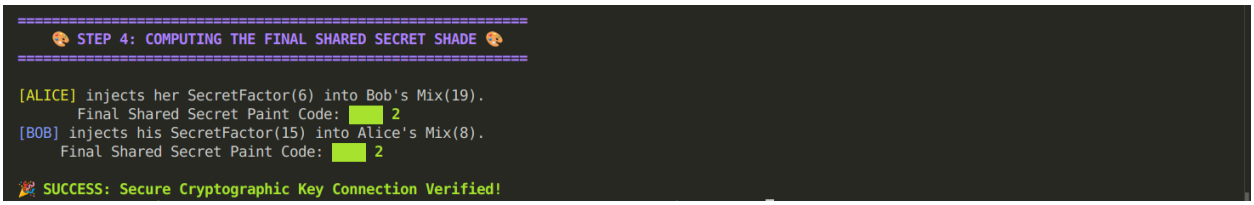
Step 1:

Step 2:

Step 3:



Step 4:



Student Reflection

This practical assignment taught me how Public Key Infrastructure (PKI) manages digital certificates to secure communications. I also implemented a Python script for the Diffie-Hellman protocol, discovering how the intractable Discrete Logarithm Problem allows secure key exchanges over open channels without revealing private keys.